pfi

Python for Fixed Income

COURSE 5B · TIME SERIES MODELS

Frequency, Lags, and the Naive Baselines

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: the rules of scoring

Development years

2 Jan 2015 to 30 Dec 2022: 2,001 dates. Every model is fitted and scored here, walk-forward.

Held-out years

3 Jan 2023 to 2 Oct 2026: 939 dates. Only the count and the dates are read, until the capstone.

Today's question

How far off is the simplest rule, and how do I score a model so that the score means something?

Three baselines, three scores, and one scoring recipe with a proof built into it.

Five series, all in bp: a miss of the last value is the day's change.

Names for a model's output

A model's output is a fitted change or a one-step value for a date inside the file. The rule it is judged against:

Allowed names

fitted change in bp

the model's one-step value

the last value carried forward

Never called

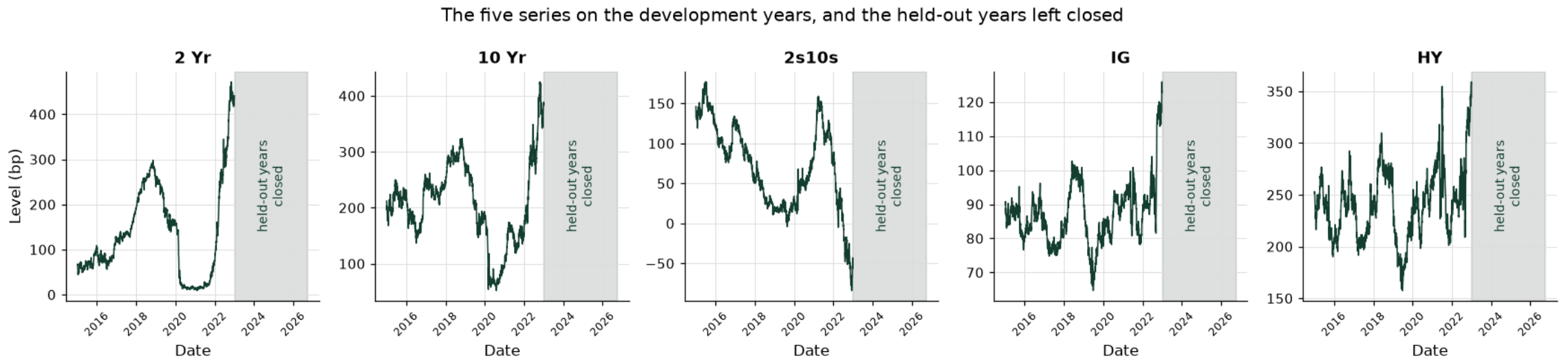
a forecast or a prediction

a projection or an outlook

a signal, a call, a view on rates

Every score is on dates already in the file, from a rule fitted on earlier dates.

The years and the five series



2,001

development dates

939

held-out dates, closed

5

series, all in bp

The full tables are deleted after the cut: no later cell can read a held-out row.

Lag by rows, not by the calendar

Weekday	10 Yr (bp)	Lag by rows	Lag by calendar
Thu 2015-01-15	177	186	186
Fri 2015-01-16	183	177	177
Mon 2015-01-19			183
Tue 2015-01-20	182	183	
Wed 2015-01-21	187	182	182

This course lags by rows

A day is a trading day, and the last value is the last one published. A calendar lag needs the holidays filled first.

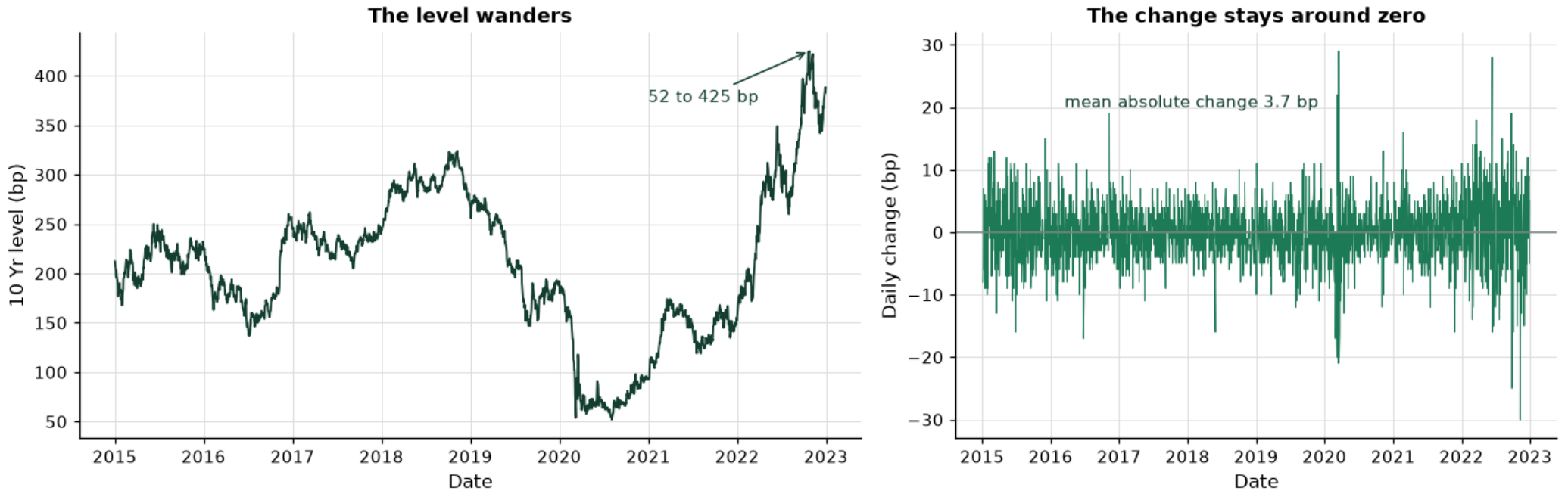
2,086 weekdays, 2,001 dates: 85 absent. Blank lags: 1 by rows, 86 by the calendar.

asfreq('B') puts the holidays back as blank rows.

How lags and changes work

The problem	A rule or a model needs the value on the row before; a model needs the change
Lag	shift(1) moves every value down one row: each date meets the value before it, the first meets nothing
Change	diff() is the level minus its lag: the first change is blank too
You control	The number of rows to move; a negative number puts a later value on an earlier date
What comes out	A column as long as the input, with that many blanks at the top, in bp here

A level wanders, a change does not



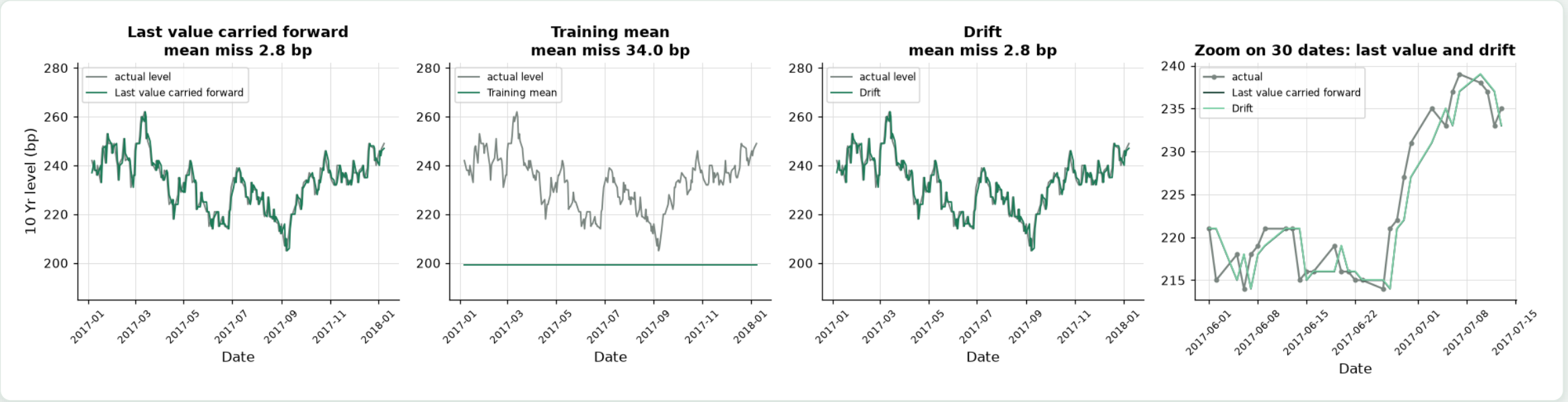
The level ran from 52 to 425 bp; the daily change has a mean absolute value of 3.75 bp and a standard deviation of 5.03 bp.

Every lag and change gets a test that it uses only earlier dates.

How the naive baselines work

The problem	A model earns its place only by beating a rule that has no model in it
Last value	A date's value is the value on the row before it
Training mean	The average of the training window's levels, fixed over the test window
Drift	The last value plus the average change in the training window, fixed the same way
What comes out	One value per test date, built only from earlier rows, ready to be scored

Three baselines on the first fold



Grey is the actual level, green the baseline. The last value hugs the level one row late; the mean is a flat line far away; drift is almost the last value.

All three use only rows before the date they are scored on.

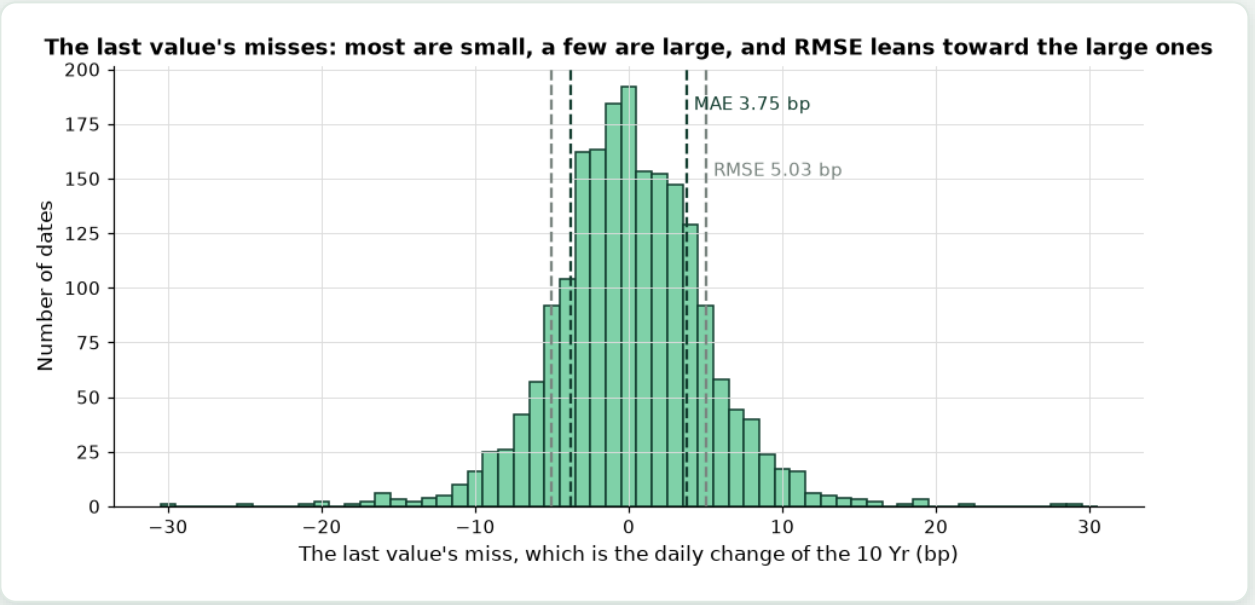
How MAE, RMSE, and MASE work

The problem	A column of misses has to become one number that can be compared
MAE	The average size of a miss, in bp
RMSE	Squares each miss first, so big days weigh more; never below MAE
MASE	A rule's MAE over the last value's MAE on the same dates: 1 is as far, below 1 is closer
You choose	Which score the decision rests on: MAE first, RMSE beside it, MASE for the baseline

Reading the scores

Baseline	MAE (bp)	RMSE (bp)	MASE
last	2.762	3.505	1.000
mean	34.022	35.727	12.318
drift	2.769	3.505	1.002

First fold of the 10 Yr, in bp. A rule that returns the last value has MASE 1.0 exactly; exactly right is 0.0; twice as far on the other side is 2.0.



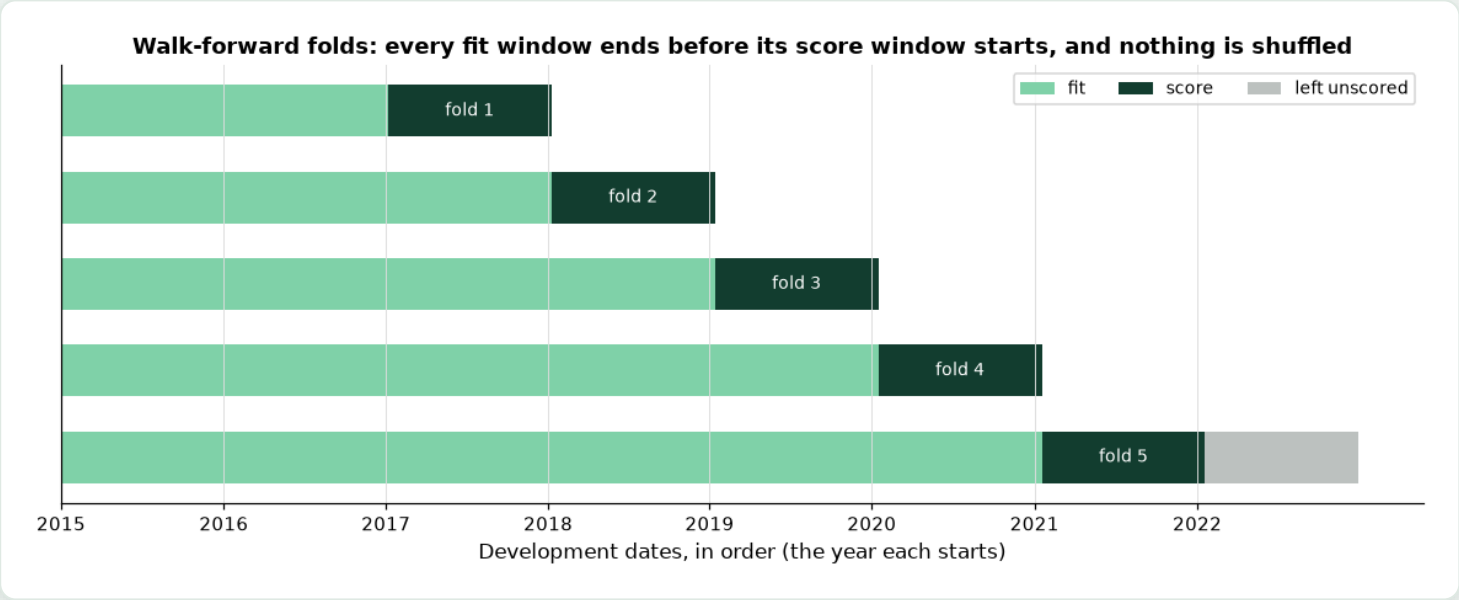
The 10 largest misses of 2,000 carry 11.1% of the squared error and 3.1% of the absolute error.

MAE 3.75 bp, RMSE 5.03 bp: RMSE listens to the large days, MAE does not.

How walk-forward scoring works

The problem	A model must be scored the way it would have run: fitted on the past, scored on what came after
Each fold	A training window of the first 504 dates, a test window of the next 252; the next fold adds the old test window to training
Never shuffled	Every training date is earlier than every test date
You control	min_train, test_size, and a gap (0 unless a target spans several dates)
What comes out	A list of folds and a score per fold; the spread of the scores says how far to trust the average

The five folds



Setting	Default	Tune first
min_train	no default	yes
test_size	no default	yes
gap	0	no

1,260 dates scored; the last 237 development dates are left unscored.

Every light bar ends where its dark bar begins. A shuffled split would paint over the whole span.

The proof that stops a cheat

fit_one(history, n_train)

Fits on history.iloc[:n_train] and returns one value per test row, the value for row t using only rows before t. A baseline, an ARIMA, and a LightGBM model all fit that shape.

The proof runs the rule again with every row from the middle of the test window on set to NaN. A value at or before that row must not move.

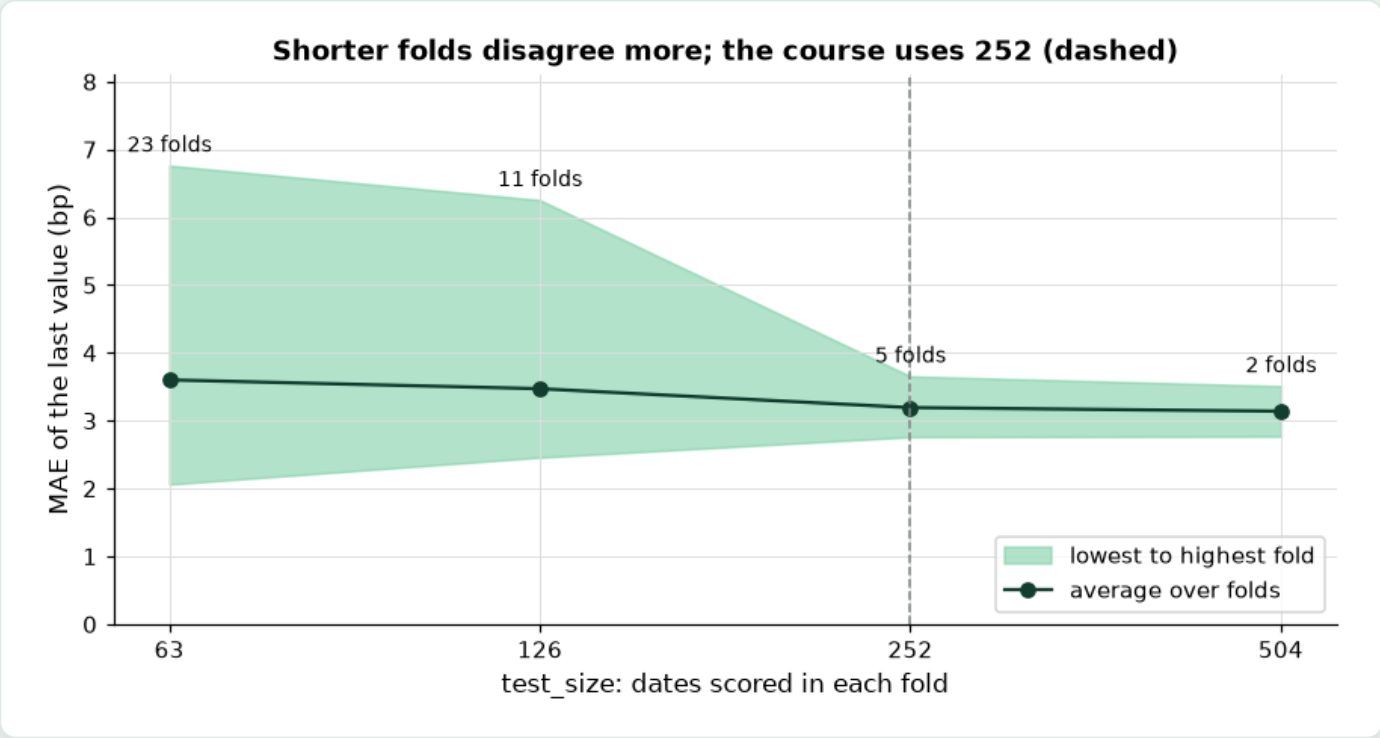
A rule that reads the date's own value

```
AssertionError:
the value for 2017-07-10
changed when later rows were removed
```

With the proof off the same rule scores 0.0 bp: perfect and meaningless. 14 tests pass.

A perfect score is how a cheat gets caught.

How long should a fold be



test_size	Folds	Lowest	Highest
63	23	2.06	6.76
126	11	2.46	6.25
252	5	2.76	3.65
504	2	2.77	3.51

Fold MAE of the last value on the 10 Yr, in bp.

The average barely moves; the spread does. The course uses 252, about a trading year.

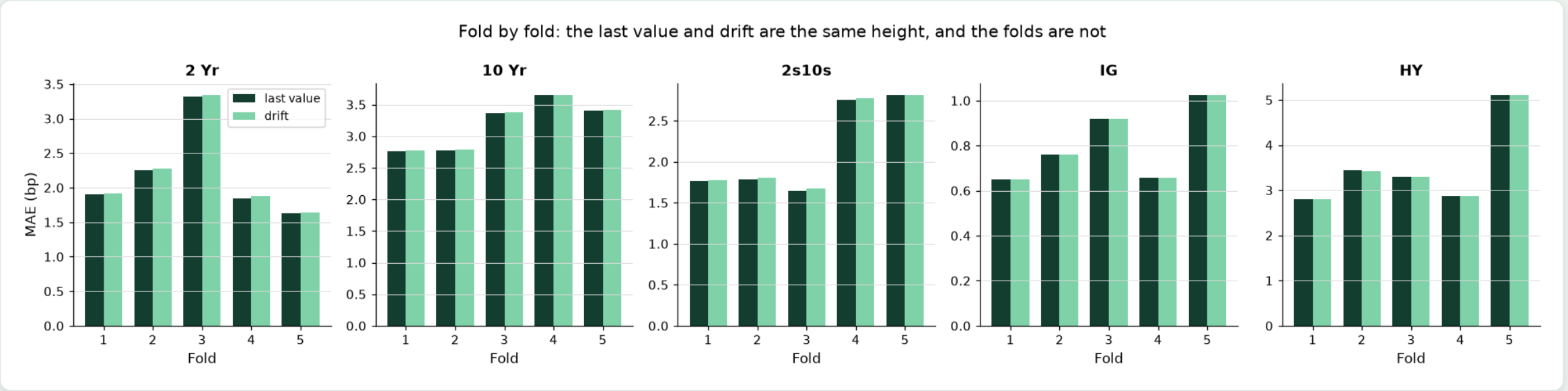
The baseline table

Series	Last value	Training mean	Drift	Drift MASE	Mean MASE
2 Yr	2.188	98.341	2.210	1.010	44.944
10 Yr	3.194	70.128	3.201	1.002	21.959
2s10s	2.155	50.605	2.168	1.006	23.485
IG	0.802	7.563	0.802	1.000	9.434
HY	3.502	29.923	3.501	1.000	8.545

MAE in bp, averaged over the five development folds, walk-forward with the proof on. Every later day sits beside this table.

Drift is the last value to a hundredth of a bp; the training mean is far worse on a level.

The folds disagree by more than the baselines do



10 Yr: the last value's folds run from 2.762 to 3.655 bp. HY: from 2.801 to 5.115 bp. Drift is within a few hundredths of the last value.

A model that beats the last value by less than the folds disagree has not beaten it.

Excel side by side

Excel side by side (tested in Excel)

`=B2  (C3, filled down)`	shift(1)
`=B3-B2  (D3, filled down)`	diff()
`=SUMPRODUCT(ABS(B506:B757-B505:B756))/ROWS(B506:B757)`	last value MAE 2.7619
`=SQRT(SUMPRODUCT((B506:B757-B505:B756)^2)/ROWS(B506:B757))`	RMSE 3.5051
`=AVERAGE($B$2:$B$505)`	training mean 199.08
`=SUMPRODUCT(ABS(B506:B757-(B505:B756+$H$2)))/ROWS(B506:B757)`	drift MAE 2.7686
`=H5/H3`	MASE 1.0024

Excel has no walk-forward loop: repeat the block with the rows moved down by 252 for each fold.

Save the day: a tag on the baselines

1. Init

The work folder, local settings, the guard

2. .gitignore

__pycache__/, .pytest_cache/, *.xlsx

3. Commits

The code, then baselines.csv

4. Tag

git tag -a baselines, with a message

```
baselines   the last value, training mean, and drift on five series, five folds
Day 2: tskit with naive_values, the scores, and walk_forward_score
```

A result with a name is a result you can return to: git show baselines:baselines.csv.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which dates fit, which dates score, what may be filled: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: shift(-1) puts every change one row early: the first date gets -8.0 bp, and the correlation with the real next change is 1.000.

Code that runs is not the same as code that is right.

What you can do now

- Set the years before any model; read only the held-out count and dates
- Lag by rows; build changes that pass a past-only test
- The last value, the training mean, and drift, drawn and scored
- MAE, RMSE, and MASE: read a MASE of 1
- Walk-forward scoring with a proof that stops a cheat

Week 1, Day 3 Stationarity and differencing